

Burp Suite Macros

What They Are · When to Use Them · How to Configure Them · Real-World Example

1. What Are Macros?

A **Macro** in Burp Suite is a saved, replayable sequence of one or more HTTP requests that Burp can automatically run *before* sending a target request. Think of it as a pre-flight routine — Burp executes the macro, extracts values from its responses, and then injects those values into your actual request.

Macros live under **Settings** → **Sessions** → **Macros** and are almost always paired with **Session Handling Rules**, which tell Burp *when* and *where* to run them.

The core problem macros solve:

Modern web apps protect their requests with values that change on every page load — CSRF tokens, nonces, anti-replay tokens, and session cookies. When you use Intruder or Repeater on such requests, the dynamic value from your original capture is already stale by the time Burp replays it. The server rejects every attempt.

Macros solve this by fetching a fresh token *automatically* before each request, so your attack always carries a valid, up-to-date value.

2. When Are Macros Used?

Scenario	Why a Macro Is Needed
Brute-forcing a login with CSRF protection	Each POST /login requires a fresh CSRF token from GET /login
Testing behind a session that expires	The macro re-authenticates so your session stays valid throughout a long Intruder run
Attacking a multi-step workflow	Step 1 must complete before step 2 is reachable — the macro runs step 1 automatically
Exploiting logic that checks a nonce	The nonce is single-use; the macro fetches a new one before every attempt
Fuzzing an endpoint that needs a valid viewstate	ASP.NET viewstates change per request; macro extracts the fresh one each time
2FA bypass labs (like the one you are working on)	The macro logs in as the victim so the /login2 endpoint has a valid session cookie ready

3. How Macros Work — The Mechanics

Macros always operate through the **Session Handling Rule** system. The rule decides *when* to trigger the macro; the macro decides *what* to do when triggered. Here is the internal flow:

Step	What Happens
1	Burp is about to send Request X (e.g., an Intruder payload attempt)
2	A Session Handling Rule fires — it matches this request by scope/tool
3	The rule runs the macro: Burp sends the macro's request(s) in the background
4	Burp extracts the configured parameter (e.g., csrf token) from the macro response
5	Burp updates Request X, replacing the stale token with the freshly extracted one

4. How to Configure a Macro — Step by Step

The following walkthrough applies to any CSRF-token scenario. Adapt the request URLs and parameter names to your specific target.

Step 1 — Record the Macro

Go to Settings → Sessions → Macros → Add.

Burp opens a request picker from your Proxy history.

Select the request that returns the CSRF token (e.g., GET /login).

Click OK. The macro is now saved with that one request.

Step 2 — Configure Parameter Extraction

In the macro editor, click on the GET /login entry, then click 'Configure item'.

Under 'Custom parameter locations in response', click Add.

Give it a name like csrf and use the response viewer to highlight the token value.

Burp will auto-fill the start/end delimiters. Click OK.

Step 3 — Create a Session Handling Rule

Go to Settings → Sessions → Session Handling Rules → Add.

Give the rule a name (e.g., 'Refresh CSRF token').

Under Rule Actions, click Add → Run a macro → select your macro.

Enable 'Update only the following parameters' and type: csrf

This tells Burp to only overwrite the csrf field, leaving everything else alone.

Step 4 — Set the Scope

Click the Scope tab within the rule.

Under 'Tools scope', tick Intruder (and/or Repeater).

Under 'URL scope', restrict to your target URL so the macro doesn't fire on every request Burp ever makes.

Step 5 — Run Your Attack

Launch your Intruder attack as normal.

Before every payload attempt, Burp silently runs the macro, grabs a fresh CSRF token, and injects it. Every request is valid.

5. Real-World Example — 2FA Bypass with Macros

This scenario maps directly to the PortSwigger **'2FA broken logic'** lab you are working on. The goal is to brute-force Carlos's 2FA code, but the `/login2` endpoint requires a valid session cookie that is tied to Carlos's partially-authenticated session. Without a macro, the session expires after a handful of attempts and every subsequent request gets redirected back to `/login`.

The Problem Without a Macro

Attempt #	What Happens
1–5	Requests reach <code>/login2</code> — valid session, codes checked
6+	Session has expired — server redirects to <code>/login</code> , attack is useless
Result	All 10,000 brute-force attempts are wasted after the first few

The Solution With a Macro

The macro re-establishes Carlos's partial session before every Intruder request by replaying the initial POST `/login` with Carlos's credentials. This sets a fresh session cookie, which the rule then injects into the `/login2` request.

Attempt #	What Happens With the Macro
Every attempt	Macro fires: POST <code>/login</code> with <code>carlos:password</code> → fresh session cookie extracted
Every attempt	Intruder uses the fresh cookie to POST <code>/login2</code> with the next 2FA code
When code matches	Server returns 302 to <code>/my-account</code> → Grep Match flags it → lab solved

Exact Configuration for This Lab

Macro Request

Record: POST `/login` with body `username=carlos&password;=montoya`

This is the request that gives Carlos a partial session after step 1 of login.

Parameter Extraction

Extract the Set-Cookie session value from the response.

Name the parameter: session

Session Handling Rule — Action

Run the macro, then update parameter: session

This injects the fresh cookie into every Intruder request.

Session Handling Rule — Scope

Tool: Intruder only

URL: `https://.web-security-academy.net/login2`

Intruder Setup

Target: POST `/login2`

Payload position: the mfa-code value

Payload list: 0000 to 9999 (numbers, min 4 digits)

Grep Match: flag responses NOT containing 'Incorrect security code'

Threads: 1 (prevents session race conditions)

6. Quick Reference — Macros vs No Macros

	Without Macro	With Macro
CSRF-protected login brute force	Every request rejected after token expires	Fresh token fetched before every attempt
Long Intruder run on auth endpoint	Session expires mid-attack	Session refreshed automatically
Multi-step flow testing	Must manually replay step 1 each time	Step 1 runs silently as part of the macro
2FA brute force (like your lab)	Session invalidated after a few tries	New partial session before every code attempt
Effort required	Manual intervention constantly needed	Fully automated — set it and walk away